

Métodos Computacionais em Física de Altas Energias

ROOT - Introdução

Marco Dias

31 de agosto de 2026

Introdução ao ROOT

ROOT é um framework orientado a objetos para análise de dados em física de altas energias.

Componentes Comuns:

- Interpretador de linha de comando (CINT)
- Histogramas e ajustes (fitting)
- Interface gráfica de usuário (GUI)
- Gráficos 2D
- Entrada/Saída (I/O)
- Classes de coleção
- Processador de scripts

Recursos:

- Site: <http://root.cern.ch>
- Manual de referência é o mais útil

Iniciando o ROOT

Iniciar uma sessão ROOT:

```
1 $ root
```

```
1 $ root -l
```

Prompt do sistema:

```
1 root [0]
```

Executar um script:

```
1 root [0] .x myScript.C
```

ou diretamente do shell:

```
1 $ root myScript.C
```

Sair:

```
1 root [0] .q
```

Iniciando o ROOT

Comandos CINT diretamente:

```
1 root [0] int i = 0
2 root [1] cout << i << endl
3 0
4 root [2]
```

O CINT faz cerca de 95% do que o C++ faz.

Formato básico:

```
1 {  
2     cout << "Hello world!" << endl;  
3 }
```

Regra do nome da função:

- Função deve ter o mesmo nome do arquivo
- Arquivo abc.cxx → void abc()

Carregando scripts:

```
1 root [] .L abc.cxx  
2 root [] bcd()
```

Inclusão de cabeçalhos:

```
1 #include <map>  
2 #include <vector>  
3 #include <string>
```

Dicas:

- ROOT pode ser instável
- Melhor executar scripts em novas sessões
- Combinar script com interface gráfica

Carregando bibliotecas:

```
1 gROOT->LoadMacro("functions.cxx");  
2 gSystem->Load("libMyEvent.so")
```

Histogramas

ROOT suporta histogramas 1D, 2D, 3D e de perfil.

Classes principais:

- TH1D - double precision, 1D
- TH1F - float precision, 1D
- TH2D, TH2F - 2D
- TH3D, TH3F - 3D

Criação:

```
1 TH1F* h = new TH1F("Name", "Title", 10, 0, 5);
```

Preenchimento:

```
1 h->Fill(value);  
2 h->Fill(value, weight);
```

Desenho:

```
1 h->Draw();  
2 h2->Draw("same"); // Sobrepor
```

Atributos:

```
1 h->SetFillColor(4); // Azul  
2 h->SetLineWidth(3);  
3 h->SetLineStyle(2);  
4 h->SetMarkerStyle(20);  
5 h->SetMarkerSize(1.5);
```


Títulos dos eixos:

```
1 h->GetXaxis()->SetTitle("x");  
2 h->GetYaxis()->SetTitle("y");  
3 h->GetZaxis()->SetTitle("z");
```

Configurações dos eixos:

```
1 h->GetXaxis()->SetRangeUser(0, 10);  
2 h->GetXaxis()->SetTitleSize(0.05);  
3 h->GetXaxis()->SetLabelSize(0.04);
```

Operações:

```
1 TH1F* sum = (TH1F*)h1->Clone();  
2 sum->Add(h2); // Adição  
3 TH1F* ratio = (TH1F*)h1->Clone();  
4 ratio->Divide(h2); // Divisão  
5 TH1F* product = (TH1F*)h1->Clone();  
6 product->Multiply(h2); // Multiplicação
```

Clonagem:

```
1 TH1F* m = (TH1F*)n->Clone();
```

Diferente de copiar apenas o ponteiro (`TH1F* m = n;`), o `Clone()` duplica a estrutura do histograma original, todos os seus binagens (conteúdo, erros), títulos, eixos e atributos de estilo (cor, estilo de linha, marcador).

Reescalonamento:

```
1 double scale = 1.0 / n->Integral();  
2 n->Scale(scale);
```

Criando um Canvas:

```
1 TCanvas* c = new TCanvas("c", "", 500, 500);  
2 c->SetFillColor(0);  
3 histo->Draw();  
4 c->Print("mygraph.C");
```

Dividindo Canvas:

```
1 canvas->Divide(2, 1);    // 2 colunas, 1 linha  
2 canvas->cd(1);           // Primeiro pad  
3 histo1->Draw();  
4 canvas->cd(2);           // Segundo pad  
5 histo2->Draw();
```

Matriz de Pads:

```
1 canvas->Divide(2, 3);    // 2x3 = 6 pads
```

Escala Log:

```
1 gPad->SetLogx();           // Escala log no eixo x
2 gPad->SetLogy();           // Escala log no eixo y
3 gPad->SetLogz();           // Escala log no eixo z
```

Pads aninhados:

```
1 TCanvas* c = new TCanvas("c", "", 800, 600);
2 TPad* pad1 = new TPad("pad1", "", 0.1, 0.5, 0.9, 0.9);
3 pad1->Draw();
4 pad1->cd();
5 histo1->Draw();
```

Gráficos (TGraph)

Criação básica:

```
1 int N = 3;  
2 double x[N] = {1, 2, 3};  
3 double y[N] = {1, 2, 3};  
4 TGraph* g = new TGraph(N, x, y);
```

Desenho:

```
1 TCanvas* c = new TCanvas("c", "", 300, 300);  
2 g->Draw("AP"); // A = eixos, P = pontos
```

Opções de desenho:

Opção	Descrição
A	Desenha eixos
C	Linha contínua
P	Polimarcadores (pontos)
L	Linha reta
*	Estrelas como marcadores
E	Barras de erro

Combinações:

```
1 g->Draw("CP");    // Linha contínua + pontos
```

Gráficos com erro:

```
1 TGraphErrors* ge = new TGraphErrors(N, x, y, ex, ey);  
2 ge->Draw("AP");
```

Ajuste de Funções (Fitting)

Ajuste básico:

```
1 histo->Fit("gaus"); // Gaussiana padrão
2 histo->Fit("pol2"); // Polinômio de grau 2
```

Funções personalizadas (TF1):

```
1 TF1* f = new TF1("f", "[0]*x + [1]", 0, 1);
2 histo->Fit("f");
```

Função definida pelo usuário:

```
1 Double_t func1(Double_t* x, Double_t* par) {
2     return par[0] * exp(-0.5 * pow((x[0]-par[1])/par[2], 2));
3 }
4 Double_t func2(Double_t* x, Double_t* par) {
5     return par[0] * TMath::Landau(x[0], par[1], par[2]);
6 }
7 Double_t fitFunction(Double_t* x, Double_t* par) {
8     return func1(x, par) + func2(x, &par[3]);}
```

Fitting - Parâmetros

Criando a função de ajuste:

```
1 TF1* theFit = new TF1("theFit", fitFunction,  
2                       fitRangeMin, fitRangeMax, nPar);
```

Definindo parâmetros iniciais:

```
1 theFit->SetParameter(0, 1.234); // Valor inicial  
2 theFit->SetParameters(1.0, 2.0, 3.0); // Múltiplos
```

Fixando parâmetros:

```
1 theFit->FixParameter(0, 1.234); // Parâmetro fixo  
2 theFit->ReleaseParameter(0); // Liberar parâmetro
```

Limites:

```
1 theFit->SetParLimits(0, 0.0, 5.0);
```


Executando o ajuste:

```
1 histo->Fit("theFit", "R"); // R = range
```

Resultados do ajuste:

```
1 TFitResultPtr r = histo->Fit("theFit", "S");  
2 double chi2 = r->Chi2();  
3 double ndf = r->Ndf();  
4 double prob = r->Prob();
```

Trees - Introdução

Trees (ntuplas) são usadas para armazenar grandes quantidades de dados para análise posterior.

Vantagens:

- Reconstrução feita apenas uma vez
- Análise rápida com diferentes cortes
- Armazenamento eficiente
- Acesso aleatório aos eventos

Criação de uma Tree:

```
1 TTree t1("t1", "Simple Tree");
2 Float_t px, py, pz;
3 Int_t ev;
4
5 // Criando branches
6 t1.Branch("px", &px, "px/F");
7 t1.Branch("py", &py, "py/F");
8 t1.Branch("pz", &pz, "pz/F");
9 t1.Branch("ev", &ev, "ev/I");
```

Preenchimento:

```
1 for (int i = 0; i < nEvents; i++) {  
2     px = ...; // Atribuir valores  
3     py = ...;  
4     pz = ...;  
5     ev = i;  
6     t1.Fill(); // Armazenar no tree  
7 }
```

Lendo uma Tree do arquivo:

```
1 TFile* file = TFile::Open("analysis.root");  
2 TTree* t1 = (TTree*)file->Get("t1");
```

Configurando branch addresses:

```
1 Float_t px, py, pz;  
2 Int_t ev;  
3 t1->SetBranchAddress("px", &px);  
4 t1->SetBranchAddress("py", &py);  
5 t1->SetBranchAddress("pz", &pz);  
6 t1->SetBranchAddress("ev", &ev);
```

Loops sobre eventos:

```
1 int entries = t1->GetEntries();  
2 for (int i = 0; i < entries; i++) {  
3     t1->GetEntry(i);  
4     cout << "Event " << ev << ": "  
5         << px << ", " << py << ", " << pz << endl;  
6 }
```

Leitura eficiente com TTree::Draw:

```
1 t1->Draw("px");  
2 t1->Draw("py:px", "pz > 10");//eixo x recebe py e o eixo y, px  
3 t1->Draw("px", "py > 5 && pz < 20");
```

Objetos personalizados na Tree:

```
1 // Definindo a classe
2 class MyEvent {
3 public:
4     Float_t pt;
5     Float_t eta;
6     Float_t phi;
7     Int_t charge;
8     // ...
9 };
```

Trees - Branch com Objetos

Criando branch com objeto:

```
1 MyEvent* event = new MyEvent();
2 TTree* tree = new TTree("tree", "Event Tree");
3
4 // split = 1 para splits automáticos
5 // bufsize = tamanho do buffer
6 tree->Branch("event", "MyEvent", &event, bufsize, split);
```

Preenchendo:

```
1 for (int i = 0; i < nEvents; i++) {
2     event->pt = ...;
3     event->eta = ...;
4     event->phi = ...;
5     event->charge = ...;
6     tree->Fill();
7 }
```

Trees - Listando e Explorando

Listando conteúdo da Tree:

```
1 TTree* tree = (TTree*)file->Get("treeName");
2 tree->Print();           // Estrutura da tree
3 tree->Scan();            // Primeiros eventos
4 tree->Scan("px:py:pz");  // Colunas específicas
5 tree->Scan("px:py:pz", "px > 10"); // Com corte
```

Listando branches:

```
1 TObjArray* branches = tree->GetListOfBranches();
2 for (int i = 0; i < branches->GetEntries(); i++) {
3     TBranch* branch = (TBranch*)branches->At(i);
4     cout << branch->GetName() << endl;
5 }
```


Listando leaves:

```
1 TTree* tree = (TTree*)file->Get("treeName");
2 TObjArray* leaves = tree->GetListOfLeaves();
3 for (int i = 0; i < leaves->GetEntries(); i++) {
4     TLeaf* leaf = (TLeaf*)leaves->At(i);
5     cout << leaf->GetName() << " : "
6         << leaf->GetTypeName() << endl;
7 }
```

Trees - Interface Gráfica (TBrowser)

Abrindo o TBrowser:

```
1 TBrowser* b = new TBrowser();
```

Interface interativa:

- Navegação por diretórios e arquivos
- Visualização de histograms e trees
- Acesso interativo a branches
- Construção de gráficos com clique

Abrindo arquivo no TBrowser:

```
1 TFile* f = TFile::Open("analysis.root");  
2 TBrowser* b = new TBrowser("Browser", f);
```

Recursos do TBrowser:

- Visualização hierárquica de objetos ROOT
- Double-click para abrir/visualizar
- Suporte a multi-arquivos

Exemplo Prático - Tree em Física

Criando uma Tree com objetos de física:

```
1 class RecoEvent {
2 public:
3     Float_t met_pt, met_phi;
4     Int_t nElectrons, nMuons, nJets;
5     Float_t electron_pt[10];
6     Float_t electron_eta[10];
7     Float_t electron_phi[10];
8     Float_t muon_pt[10];
9     Float_t muon_eta[10];
10    Float_t muon_phi[10];
11    Float_t jet_pt[20];
12    Float_t jet_eta[20];
13    Float_t jet_phi[20];
14 };
```

Exemplo Prático - Tree em Física

Criando branches:

```
1 RecoEvent* ev = new RecoEvent();
2 TTree* tree = new TTree("reco", "Reconstruction Tree");
3 tree->Branch("reco", "RecoEvent", &ev, 64000, 1);
```

Preenchendo:

```
1 for (int i = 0; i < nEvents; i++) {
2     // Reconstrução
3     ev->met_pt = ...;
4     ev->nElectrons = ...;
5     for (int j = 0; j < ev->nElectrons; j++) {
6         ev->electron_pt[j] = ...;
7     }
8     tree->Fill();
9 }
```

Análise com TTree::Draw:

```
1 // Histograma simples
2 tree->Draw("electron_pt");
3
4 // Corte básico
5 tree->Draw("muon_pt", "muon_pt > 10");
6
7 // Corte com múltiplas condições
8 tree->Draw("jet_pt", "nJets > 2 && met_pt < 50");
9
10 // Gráfico 2D
11 tree->Draw("electron_eta:electron_phi");
12
13 // Expressões matemáticas
14 tree->Draw("TMath::Sqrt(px*px + py*py + pz*pz)");
```

Análise com TTree::MakeClass:

```
1 tree->MakeClass("MyAnalyzer");
```

Cria um arquivo MyAnalyzer.C com:

- Estrutura Loop() para processamento
- Todos os branches como variáveis
- Template para seleção de eventos

Interface Gráfica do ROOT

Componentes da Interface Gráfica:

- TBrowser - Navegação de arquivos e objetos
- TCanvas - Área de desenho
- TEditor - Edição de objetos
- TTimer - Temporização
- Menus - Interação com objetos

Usando o TEditor:

```
1 // Criar canvas e objeto
2 TCanvas* c = new TCanvas();
3 TH1F* h = new TH1F("h", "Histogram", 100, 0, 10);
4 h->Draw();
5
6 // Abrir editor
7 TEditor* editor = new TEditor(c);
```

Abrindo múltiplos arquivos:

```
1 TFile* f1 = TFile::Open("data1.root");  
2 TFile* f2 = TFile::Open("data2.root");  
3 TBrowser* b = new TBrowser();
```

Visualizando objetos:

- Double-click em TH1 → histograma
- Double-click em TTree → estrutura
- Double-click em TGraph → gráfico
- Double-click em diretório → navegação

Ações comuns no TBrowser:

- Desenhar: arrastar para canvas
- Ver dados: botão "Scan"
- Fazer histograma: selecionar branch e "Draw"
- Salvar: botão "Save" no canvas

Exemplo de fluxo de trabalho:

- 1 Abrir TBrowser
- 2 Navegar para arquivo .root
- 3 Abrir Tree, explorar branches
- 4 Fazer histogramas interativos
- 5 Exportar para script

Salvando e Exportando

Salvando canvas:

```
1 c->SaveAs("plot.pdf");  
2 c->SaveAs("plot.png");  
3 c->SaveAs("plot.C");
```

Salvando script da figura:

```
1 c->Print("plot.C");
```

Exportando histogramas:

```
1 h->SaveAs("hist.root");  
2 h->SaveAs("hist.txt");
```

Criando scripts de análise:

```
1 // Do TBrowser: File -> Save Canvas As -> C++ Script  
2 // Ou usar:  
3 c->Print("analysis_script.C");
```

Histogramas 2D:

```
1 TH2F* h2 = new TH2F("h2", "Title", 50, 0, 10, 40, -5, 5);  
2 h2->Fill(x, y);  
3 h2->Draw("COLZ"); // Com barra de cores
```

Histogramas de Perfil:

```
1 TProfile* prof = new TProfile("prof", "Profile", 50, 0, 10);  
2 prof->Fill(x, y);  
3 prof->Draw();
```

Histogramas 3D:

```
1 TH3F* h3 = new TH3F("h3", "Title", 20, 0, 10, 20, 0, 10, 20, 0, 10);  
2 h3->Fill(x, y, z);  
3 h3->Draw("BOX2");
```

TMultiGraph - Múltiplos gráficos:

```
1 TMultiGraph* mg = new TMultiGraph();  
2 mg->Add(g1);  
3 mg->Add(g2);  
4 mg->Add(g3);  
5 mg->Draw("AP");
```

Gráficos de erros assimétricos:

```
1 TGraphAsymmErrors* ge = new TGraphAsymmErrors();  
2 ge->SetPoint(i, x, y);  
3 ge->SetPointError(i, xl, xh, yl, yh);  
4 ge->Draw("AP");
```

Gráficos de barras:

```
1 TH1F* h = new TH1F("h", "Title", 10, 0, 10);  
2 h->FillRandom("gaus");  
3 h->Draw("BAR");
```

Gráficos de parâmetros de ajuste:

```
1 TF1* f = new TF1("f", "[0]*x + [1]", 0, 10);  
2 histo->Fit(f);  
3 g->Draw("AP");  
4 f->Draw("same");
```

Criando legenda:

```
1 TLegend* leg = new TLegend(0.7, 0.7, 0.9, 0.9);  
2 leg->AddEntry(h1, "Data", "l");  
3 leg->AddEntry(h2, "MC", "l");  
4 leg->AddEntry(h3, "Background", "f");  
5 leg->Draw();
```

Adicionando texto:

```
1 TLatex* tex = new TLatex();  
2 tex->SetTextSize(0.04);  
3 tex->DrawLatex(0.5, 0.5, "CMS");  
4 tex->DrawLatex(0.2, 0.8, "#sqrt{s} = 13 TeV");  
5  
6 // Texto com símbolos  
7 tex->DrawLatex(0.5, 0.6, "#mu_{#eta}");  
8 tex->DrawLatex(0.5, 0.4, "E_{T}^{miss}");
```

Linhas e formas:

```
1 TLine* line = new TLine(x1, y1, x2, y2);  
2 line->Draw();  
3  
4 TArrow* arrow = new TArrow(x1, y1, x2, y2);  
5 arrow->Draw();  
6  
7 TLatex* lab = new TLatex(x, y, "#chi^2 / ndf = 1.2");  
8 lab->Draw();
```

Macro de Análise Completa

```
1 void analyze() {  
2     // Abrir arquivo  
3     TFile* f = TFile::Open("data.root");  
4     TTree* tree = (TTree*)f->Get("reco");  
5  
6     // Criar histogramas  
7     TH1F* h_mass = new TH1F("mass", "Invariant Mass", 100, 0, 200);  
8     TH1F* h_pt = new TH1F("pt", "Transverse Momentum", 100, 0, 100);  
9  
10    // Branches  
11    Float_t mass, pt;  
12    tree->SetBranchAddress("mass", &mass);  
13    tree->SetBranchAddress("pt", &pt);  
14  
15    // Loop  
16    for (int i = 0; i < tree->GetEntries(); i++) {  
17        tree->GetEntry(i);  
18        if (pt > 20) {  
19            h_mass->Fill(mass);  
20        }  
21        h_pt->Fill(pt);  
22    }
```


Macro de Análise Completa

```
1 // Ajuste
2 TF1* fit = new TF1("fit", "[0]*exp(-0.5*((x-[1])/[2])**2)", 0, 200);
3 fit->SetParameters(100, 91, 5);
4 h_mass->Fit(fit, "R");
5
6 // Desenhar
7 TCanvas* c = new TCanvas("c", "", 900, 600);
8 c->Divide(2, 1);
9 c->cd(1);
10 h_pt->Draw();
11 c->cd(2);
12 h_mass->Draw();
13 fit->Draw("same");
14
15 // Salvar
16 c->SaveAs("analysis.pdf");
17 }
```